

A BDI-LLM Hybrid Multi-Agent System for DeliverooJS

Davide Senette
email: davide.senette@studenti.unitn.it, mat: 267070

Davide Sarti
email: davide.sarti@studenti.unitn.it, mat: 267894

Autonomous Software Agents Course
17 June 2026

1 System Overview

Agent A — the LLM agent. Runs an *embedded* autonomous BDI loop (`startAutonomousBDI`), identical to Agent B’s, and additionally listens on the chat channel: each message is interpreted by an LLM and executed through a tool-call loop (Section 6). The embedded BDI keeps harvesting with zero model calls while the LLM merely reads/queries state, and is paused only when a tool takes exclusive control of the agent’s body (movement, delegated collection, rendezvous).

Agent B — the BDI agent. A pure reactive BDI agent with no LLM in its loop: it scores and executes intentions autonomously (Section 4) and can be *preempted* by directives from Agent A (Section 7).

Both agents instantiate `createBeliefState()` (Section 2) independently; the two belief states are kept consistent by an explicit synchronization channel for persistent rules, not by sharing memory. The design separates four recurring concerns:

- **Deliberation** — option generation and scoring (`bdi/options.js`, `bdi/bdiAgent.js`), independent of how an intention is eventually carried out.
- **Action execution** — the plan layer (`actions/actions.js`), which turns a predicate into pathfinding and game-socket calls, falling back to PDDL when needed.
- **Coordination** — a directive/status protocol (`bdi/coordination.js`, `llm/coordinator.js`) layered on top of, but decoupled from, the scoring loop.
- **Shared beliefs** — one schema (`beliefs/beliefState.js`) consumed identically by scoring, navigation, the LLM tools and the PDDL problem builder.

Figure 1 shows the resulting data and control flow.

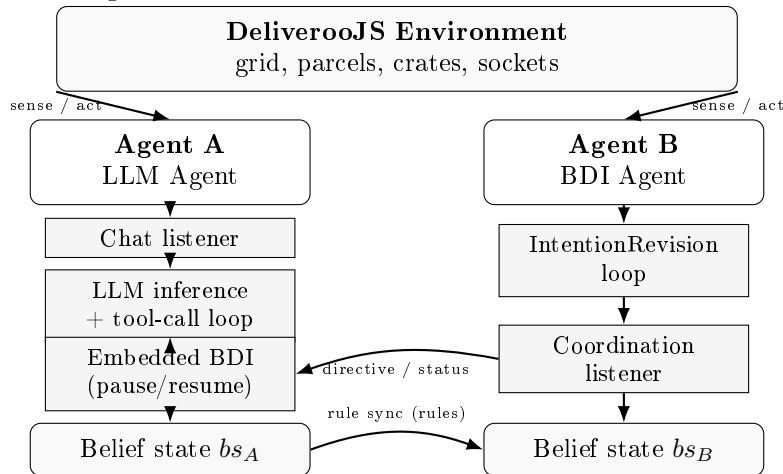


Figure 1: Overall system architecture: two agents with structurally identical belief states, connected by a rule-synchronization channel and a directive/status coordination channel. The LLM agent’s embedded BDI is the same code path as the standalone BDI agent’s loop.

2 Belief State

`beliefs/beliefState.js` defines `createBeliefState()`, the single source of truth read by option generation, scoring, navigation, the LLM tools and the PDDL problem builder. No module keeps a parallel copy

of world state; everything is derived from this object on demand. Table 1 summarizes its top-level sections.

Section	Content
me	{id, name, x, y, score} — identity and current position.
parcels, crates, agents	Maps of sensed world objects keyed by id, refreshed on every sensing event.
map	tiles, deliveryTiles, spawnTiles, pushableTiles (type-5/"5!" crate-receiving tiles), deliveryDistanceMap, spawnDistanceMap (BFS tables, Section 5), grid.
config	observationDistance, movementDuration, playerCapacity, parcelDecayingEvent, parcelGenerationEvent, maxParcels — server-declared parameters.
timing	decayTicks (global counter of observed decay events) and decayPerStep (EMA of ticks per completed move, Section 4).
rules	Persistent strategy rules (Table 2), set only by the LLM agent's tools but mirrored onto the BDI agent's copy.
coordination	active, queue, current, waiting, sendStatus, lastActivityMs — directive-mode state (Section 7).
carry	{count, campSteps} — cached carried-parcel count and tiles patrolled while carrying in the current camp episode.
lastParcelHint	{x, y, ts} of the last tile where a parcel was <i>actually picked up</i> (not merely sighted); anchors camping.

Table 1: Top-level sections of the belief state.

Field	Semantics
stackSize	Array of {mode, count, met, unmet}; compatible constraints (e.g. at_least 2, at_most 5) coexist; conflicts are resolved at write time.
parcelValueRules	Array of {minReward, maxReward, mult, delta} value-band remaps applied at delivery time.
penaltyDeliveries, preferredDeliveries	"x,y" → {penalty} / {reward} maps for specific delivery tiles.
deliveryMultipliers	"x,y" → multiplier map for specific delivery tiles.
penaltyTiles	"x,y" → {penalty} map consumed by A* as a soft cost (<i>not</i> a hard block).
rendered	Human/LLM-readable string derived from the structured fields, kept current by the rule tools.
onChange	Optional hook (wired by the LLM agent) fired after any rule mutation, used to push a full rule snapshot to the partner.

Table 2: Fields of `bs.rules`. All are soft preferences: they shift a score but, by construction (Section 4), can never strand the agent without a viable delivery or a passable route.

Object permanence. Sensing is local and intermittent: a parcel or agent leaving observation range simply stops appearing in updates. Belief-update paths treat this absence as negative evidence with a short time-to-live rather than as silence: an agent's last known position (`AGENT_MEMORY_TTL_MS`, 3 s) feeds the race-discount and soft-obstacle computations only briefly before being dropped, and the same TTL discipline governs the avoided-tile memory used in navigation (Section 5). The spawn-tile staleness heat map driving `explore` (Table 6) ticks once per sensing snapshot rather than per move, so it keeps cooling/heating tiles even while the agent stands still (camping, parked on a coordination wait) instead of freezing.

3 Option Generation

`optionsGeneration(agent, bs)` (`bdi/options.js`) turns the current belief state into a set of candidate predicates and inserts them into the intention queue in one batched call. It is purely generative: it decides what is *reachable*, never what is *worth doing* — valuation is the exclusive responsibility of `intentionScore` (Section 4).

`agent.pushBatch(options)` filters out predicates already cooled-down or queued, inserts the rest, and re-sorts the queue once regardless of batch size, avoiding $O(n)$ re-sorts for n new options.

`optionsGeneration` returns immediately when `bs.coordination.active` is true, so ordinary scored options never compete with or preempt the active coordination intention (Section 7).

4 Scoring and Intention Revision

4.1 The IntentionRevision loop

`IntentionRevisionRevise` (`bdi/bdiAgent.js`) maintains an intention queue and a failed-intention cooldown pool (predicate → {addedAtMs, cooldownMs}). The cooldown backs off per predicate — base `FAILED_INTENTION_RETRY_MS`=1 s, doubling on each consecutive failure (`FAILED_INTENTION_BACKOFF_FACTOR`=2) up to `FAILED_INTENTION`

Predicate	Generated when	Source
<code>go_pick_up x y id</code>	a free parcel has a finite pickup-route distance, and carried count < effective capacity	one per visible free parcel
<code>go_drop_off x y</code>	at least one parcel is carried	one per delivery tile reachable from the current cell
<code>camp</code>	<code>AGENT_CONFIG.behavior.camp</code> is enabled	single option, validity decided entirely in scoring
<code>explore</code>	always, as the idle fallback pushed directly by the loop	not generated here; pushed by <code>IntentionRevision.loop()</code> when the queue is empty

Table 3: Predicates produced by `optionsGeneration` (`explore` is pushed separately by the idle branch of the revision loop, Section 4).

`MAX_RETRY_MS=8s` — so a persistently blocked target stops retry-storming; the streak resets on the predicate’s next success, or automatically once it goes $2\times$ the cap (16s) without a further failure, so a resolved block does not leave a stale escalated cooldown behind. Each pass of `loop()`:

1. Idles if **paused** (an embedded owner, e.g. the LLM mission handler, is holding the body) or runs one coordination step if `coordination.active`.
2. Re-queues failed intentions whose cooldown has elapsed.
3. Drops the head intention if it has become structurally invalid (parcel already carried/gone, or no longer carrying anything for a queued delivery).
4. Otherwise `achieve()`s the head intention, classifying the outcome as *completed*, *preempted*, *failed* or *stopped*, and reacts accordingly (Section 4.4).
5. Pushes **explore** when the queue is empty.

4.2 Score computation

`intentionScore(predicate)` is the single authority on a predicate’s worth, computed fresh from `bs` at scoring time.

Decay factor. Reward loss is priced through one quantity, the EMA-estimated reward lost per carried parcel per tile traveled:

$$\text{factor} = \text{decayPerStep} = \alpha \cdot \text{ticks}_t + (1 - \alpha) \cdot \text{decayPerStep}_{t-1}, \quad \alpha = 0.15 \quad (1)$$

seeded by the config-derived prior `movementDuration/decayInterval` and sampled purely from server events (decay ticks observed between two consecutive move acknowledgements), so it is immune to client load, network latency or server lag: a slower step necessarily contains proportionally more ticks.

Drop-off score. For `go_drop_off x y` with route distance d (read from `deliveryDistanceMap`) and carried parcels $p \in P$:

$$\text{projected}_p = \max(0, \text{reward}_p - d \cdot \text{factor}) \quad (2)$$

$$\text{banked}_p = \text{projected}_p \cdot \text{mult}_v + \text{delta}_v \quad (\text{value-band remap, Section 4.2.1}) \quad (3)$$

$$\text{sumBanked} = \sum_{p \in P} \text{banked}_p \quad (4)$$

$$\text{scoreAfterStack} = \text{sumBanked} \cdot \text{mult}_s + \text{delta}_s \quad (\text{stack delivery modifier at } |P|) \quad (5)$$

$$S_{\text{drop}}(x, y) = \max(0, \text{scoreAfterStack} \cdot m(x, y) + r(x, y) - \pi(x, y)) \quad (6)$$

where m, r, π are the tile’s delivery multiplier, preferred reward and penalty (`adjustDeliveryScore`).

Pickup score. For `go_pick_up` of a new parcel n with full route distance d (pickup leg + nearest-delivery leg) and carried parcels P :

$$\text{raceProb}(n) = \frac{1}{1 + \exp((d_{\text{me} \rightarrow n} - d_{\text{opp} \rightarrow n})/\tau)}, \quad \tau = \text{RACE_DISCOUNT_TAU} = 2 \quad (7)$$

$$\text{newDelivered} = \left(\max(0, \text{reward}_n - d \cdot \text{factor}) \cdot \text{mult}_v + \text{delta}_v \right) \cdot \text{raceProb}(n) \quad (8)$$

$$\text{carriedDelivered} = \sum_{p \in P} \left(\max(0, \text{reward}_p - d \cdot \text{factor}) \cdot \text{mult}_{v,p} + \text{delta}_{v,p} \right) \quad (9)$$

$$S_{\text{pick}}(n) = (\text{carriedDelivered} + \text{newDelivered}) \cdot \text{mult}_s(|P|+1) + \text{delta}_s(|P|+1) \quad (10)$$

The stack modifier in Eq. 10 is evaluated *optimistically* at the resulting count $|P|+1$: gathering toward a target is priced with the rule’s **met** modifier, and only a pickup that *overshoots* an **exactly/at_most** cap takes the **unmet** modifier (**stackPickupModifier**), so a sufficiently rich parcel can still justify breaking the target stack. The race discount is applied *after* the value-band remap, never before: discounting first could push a high-value parcel into a “worth 0” band the moment any agent contests it.

Camp score. **camp** is valid only while carrying ($|P| > 0$), the harvest hint is “hot”, capacity remains, and no delivery tile is within **CAMP_NEAR_DELIVERY_TILES** = 8 (loitering near a delivery is never worth it: quick cycles win). Patience is primarily spatial, then scaled by the server’s declared spawn rate:

$$\text{patienceMs} = \text{clamp}(\mu \cdot 1000 \cdot (1.37^{n_{\text{adj}}} - 1), 0, \mu \cdot 8000), \quad n = \min(n_{\text{adj}}, 10) \quad (11)$$

forced to 0 when $n_{\text{adj}} < 4$ or the server declares infinite parcel generation, with n_{adj} the number of spawn tiles within Manhattan radius 3 of the harvest anchor (the constants are **CAMP_PATIENCE_MIN_TILES**=4 and **CAMP_PATIENCE_SATURATION_TILES**=10) and $\mu = \min(\text{CAMP_SPAWN_RATE_MAX_MULT}, \max(1, \text{CAMP_SPAWN_RATE_NEUTRAL_MS}/\text{parcelGenerationEvent}))$, **CAMP_SPAWN_RATE_MAX_MULT**=1.3 and **CAMP_SPAWN_RATE_NEUTRAL_MS**=2000 ms, so a map that spawns faster than the 2 s neutral rate stretches both the growth curve and its ceiling, making a hot pocket worth camping longer. While hot, the camp loss budget bounds how long camping is worth it relative to delivering now:

$$\text{horizon} = \frac{\max(\text{CAMP_LOSS_BUDGET_FRACTION} \cdot \text{totalReward}, \text{stackCampGain})}{\text{factor} \cdot |P|} \quad (12)$$

where **stackCampGain** is the extra delivery value reachable by gathering more parcels up to an active stack-rule target (0 when no rule, or when the cap is already exceeded and cannot be undone by waiting). Camp is invalid once **campSteps** $\geq$ horizon. When valid:

$$S_{\text{camp}} = \text{totalReward} + \text{CAMP_INCENTIVE}, \quad \text{CAMP_INCENTIVE} = 0.02 \quad (13)$$

which outranks a plain delivery of the same load (so the agent tops up first) but stays below any real pickup ($\text{totalReward} + \text{reward}_n$), so a visible parcel is still grabbed first.

Contested-pocket tie-break. Camp validity also depends on the partner’s position, via a shared radius

$$\text{campRadius}(bs) = \lceil \text{observationDistance} \rceil, \quad (14)$$

falling back to **CAMP_PATROL_RADIUS**=5 tiles without a declared observation distance — reused verbatim by the camp anchor/patrol footprint (Section 5) and the exploration de-conflict below. When the partner is within this radius of the harvest anchor, only the agent with the *lower* game score keeps camping (ties broken by id), so both sides reach the same verdict with no message exchange; the loser’s **camp** scores -1 , handing off to delivery or exploration as an ordinary cold pocket would. The verdict cannot flip mid-camp, since camping freezes the camper’s own score while the partner’s can only grow.

Exploration score. A flat floor, $S_{\text{explore}} = \text{EXPLORATION_INCENTIVE} = 0.01$, strictly below camp so an idle agent prefers to camp a hot pocket over wandering, but high enough to keep the agent moving when nothing else scores positively.

4.2.1 Persistent-rule modifiers

bdi/ruleScoring.js implements the rule effects referenced above as pure functions of **bs.rules**, kept independent of the scoring loop so they cannot silently diverge from it:

- **Parcel value bands.** Each rule $\{\text{minReward}, \text{maxReward}, \text{mult}, \text{delta}\}$ contributes its modifier when a value v lies in $[\text{minReward}, \text{maxReward}]$ (open bounds), combined multiplicatively/additively across overlapping rules: $v' = v \cdot \prod \text{mult}_i + \sum \text{delta}_i$.
- **Stack-size combination.** **combineStack** multiplies every active rule’s chosen modifier (**met** or **unmet**, selected per-context) and sums the deltas; two rules *conflict* (and the older is dropped on write) exactly when their satisfied-count ranges — $[N, \infty)$ for **at_least**, $[1, N]$ for **at_most**, $[N, N]$ for **exactly** — do not overlap.
- **Delivery-tile preferences.** $\text{adjusted} = \max(0, \text{base} \cdot m + r - \pi)$ (Eq. 6); the floor at 0 is what keeps a penalty a pure *reordering* device rather than a way to forbid the agent’s only path to banking reward.

Table 4 collects the numeric constants governing scoring and preemption.

Constant	Value	Role
HOARD_CAP	10	Hard ceiling on carried parcels regardless of server-declared capacity.
RACE_DISCOUNT_TAU	2 tiles	Sigmoid temperature in Eq. 7.
PREEMPTION_HYSTERESIS	0.25	Relative margin a challenger must clear (Section 4.3).
PREEMPTION_EPSILON	0.001	Additive margin guarding against float-equality ties.
CAMP_NEAR_DELIVERY_TILES	8	Distance below which camping is disabled in favor of delivering.
CAMP_LOSS_BUDGET_FRACTION	0.05	Fraction of carried reward camping may bleed to decay.
EXPLORATION_INCENTIVE	0.01	Flat exploration floor.
CAMP_INCENTIVE	0.02	Flat margin by which camp outranks an equal-value delivery.
FAILED_INTENTION_RETRY_MS	1000 ms	Base cooldown before a failed predicate is re-probed (Section 4).
FAILED_INTENTION_BACKOFF_FACTOR	2	Per-streak doubling factor applied to the cooldown above.
FAILED_INTENTION_MAX_RETRY_MS	8000 ms	Cap on the backed-off cooldown.
CAMP_PATROL_RADIUS	5 tiles	Fallback campRadius(<i>bs</i>) when the server declares no observation distance.

Table 4: Scoring and preemption constants (`utils/constants.js`, `bdi/options.js`).

4.3 Hysteresis preemption

`sortQueueByScore()` re-scores every queued intention, drops any with score ≤ 0 (so a hard-forbidden delivery, floored to 0 by Eq. 6, naturally leaves the queue), and re-sorts by descending score. Switching the *running* intention is not free — a half-walked route has a real sunk cost — so a challenger must clear a hysteresis band:

$$S_{\text{challenger}} > \begin{cases} S_{\text{running}} (1 + h) + \varepsilon & \text{if } S_{\text{running}} > 0 \\ 0 & \text{if } S_{\text{running}} \leq 0 \end{cases} \quad (15)$$

with $h = 0.25$ and $\varepsilon = 0.001$ (Table 4); an already-invalid running intention is abandoned immediately. This naturally protects deliveries in final approach: the drop-off score *peaks* right before the tile (route distance $\rightarrow 0$), so the interruption bar is highest exactly where switching would waste the most progress. A `committedManeuver` flag (set during a PDDL crate-push, Section 8) unconditionally disables preemption, since restarting a partial push from scratch would waste the detour already taken.

When preemption fires, the running intention is `stop(STOP_REASON_PREEMPTION)`’d; its `achieve()` promise rejects with a tagged *preempted* error, and the loop **requeues** a fresh `Intention` for the same predicate to compete again next sort.

4.4 Failure handling

Outcome	Handling
<i>completed</i>	Intention removed from the queue; no special action.
<i>preempted</i>	Re-queued as a fresh intention (Section 4.3); never enters the cooldown pool.
<i>failed</i>	Added to the failed-intention pool for <code>FAILED_INTENTION_RETRY_MS</code> ; if the failed predicate was <code>go_drop_off</code> , <code>#pushNextNearestDelivery</code> immediately queues the next-nearest <i>other</i> delivery tile so the agent is never left with zero delivery options while parcels keep decaying.
<i>stopped</i>	A clean, non-preemption stop (e.g. a manual pause or a propagated <code>["stopped"]</code>); removed without cooldown or replacement.

Table 5: Intention outcomes and their effect on the queue.

Figure 2 summarizes the full cycle.

5 Plans and Navigation

`executePredicate(actions/actions.js)` is the one entry point translating a predicate into game-socket calls; it is shared verbatim by the autonomous loop, the coordination layer and the LLM’s delegated tools.

Navigation principles.

- **A* with mixed hard/soft obstacles.** Walls and crates are hard; other agents are hard only within `SOFT_OBSTACLE_HARD_RADIUS=2` tiles of the start (they are genuinely in the way *now*), and merely add `AGENT_SOFT_PENALTY=7.5` to the edge cost otherwise (they will likely have moved by the time the agent arrives).

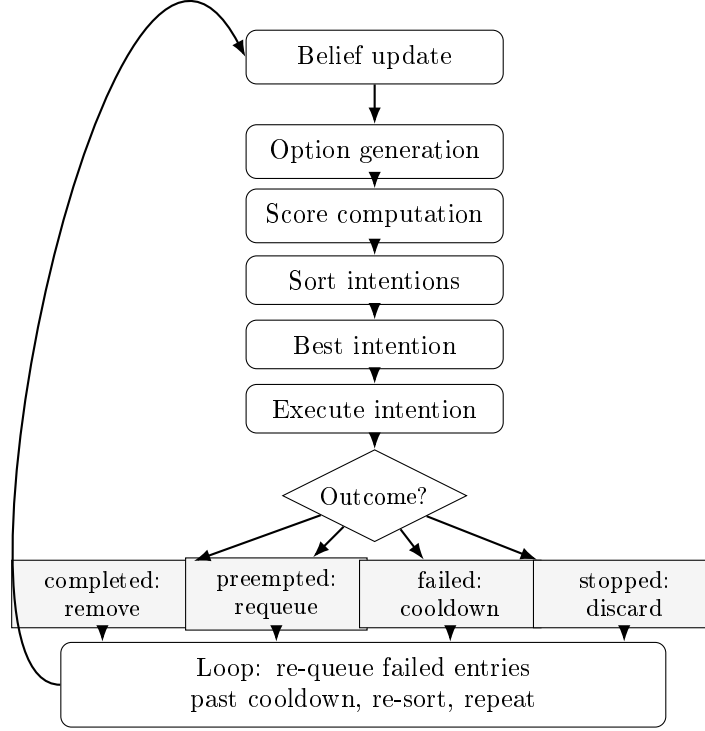


Figure 2: Intention revision cycle. The sort step (Eq. 15) is where hysteresis preemption is decided; the four outcomes feed back into the same loop via the failed-intention pool and the delivery fallback of Section 4.4.

Predicate	Behavior
<code>go_to x y</code>	A*-routes to (x, y) , replanning on a blocked step (up to <code>MAX_REPLANS=3</code>) and falling back to PDDL when a crate is the sole blocker.
<code>go_pick_up x y id</code>	<code>go_to</code> then <code>pickup()</code> ; records <code>lastParcelHint</code> on a successful grab.
<code>go_drop_off x y</code>	<code>go_to</code> then <code>putdown()</code> , which itself credits <code>bs.metrics.deliveredParcels</code> from the server ack on genuine delivery tiles — so opportunistic and PDDL-driven drops are counted identically (a handoff drop on a plain tile banks nothing).
<code>explore</code>	<code>go_to</code> on the least-recently-visited reachable spawn tile (<code>findCellsToExplore</code> , travel-cost/staleness weighted).
<code>camp</code>	Patrols the perimeter of the spawn cluster around the harvest anchor (Section 4), charging <code>carry.campSteps</code> .
<code>go_near x y maxDist</code>	<code>go_to</code> on the closest reachable tile inside the Manhattan diamond of radius <code>maxDist</code> around (x, y) ; used by rendezvous.
<code>wait signal timeoutMs</code>	Blocks until <code>bs.coordination.waiting</code> is cleared by an incoming <code>signal</code> message, or until an optional timeout; default is unbounded.

Table 6: Plan predicates dispatched by `executePredicate`.

- **Soft penalties from rules.** Entering a tile in `bs.rules.penaltyTiles` costs extra (added to `BASE_STEP_COST=1`) but is never refused, so an LLM navigation rule can steer the route without ever stranding the agent.
- **Adaptive timeout.** `timeout = max(GO_TO_TIMEOUT_MS, |path| · movementDurationMs · GO_TO_TIMEOUT_SAFETY_FACTOR)`, with `GO_TO_TIMEOUT_SAFETY_FACTOR=3`, so short legs fail fast and long legs are not spuriously timed out.
- **avoidTiles.** A tile that just refused a move is hard-blocked for `AVOID_TENURE_MOVE_FACTOR=3` move-durations, forcing the replanner off a proven choke; the memory expires quickly enough that a cleared corridor is retried while still in sensing range.
- **Opportunistic pickup and delivery.** `opportunisticActions()` runs after every step: it grabs a free, rule-eligible parcel on the just-entered tile (zero detour) and auto-banks carried parcels on a delivery tile, unless a stack rule is unmet or the tile is penalised — suspended while `coordination.active`. Both are best-effort: a failed socket ack is swallowed rather than thrown, so a transient hiccup never aborts the walking intention.
- **BFS distance maps.** `deliveryDistanceMap` and `spawnDistanceMap` are precomputed once per tile (multi-source shortest-path tables) so route-distance lookups used throughout scoring (Eqs. 6–10) are $O(1)$ instead of re-running BFS/A* on every score evaluation.
- **Penalty-aware travel-cost map.** `travelCostMap()` runs a single Dijkstra pass (`pathfinding/dijkstra.js`) from the agent’s position, giving $O(1)$ penalty-inclusive reachability/cost lookups reused by `goNear`, `explore`’s ranking (`findCellsToExplore`) and the camp anchor’s cluster-centroid choice (`clusterCenter`) — replacing one A*/BFS probe per candidate. An unreachable candidate is dropped rather than scored as NaN; without a usable map, callers fall back to plain geometric distance.
- **Exploration/partner de-conflict.** `explore` demotes (never drops) spawn-tile candidates within the partner’s `campRadius(bs)` (Section 4) to the tail of the ranking, so the two agents spread over separate pockets while still falling back to a watched tile if nothing else is reachable.
- **Crate detection.** `crateBlocksRoute()` plans twice — once respecting crates, once with `ignoreCrates:true` — and concludes a crate is the blocker only when the first fails and the second succeeds, avoiding false positives from an agent merely orbiting a temporarily occupied choke.
- **PDDL invocation.** On a confirmed crate block, `pddlReach` sets `bs.committedManeuver=true`, solves and executes a push plan (Section 8), and clears the flag in a `finally`, regardless of outcome.

6 The LLM Layer

6.1 Architecture

`startLLMagent(socket, bs, llmState, actions)` wires three independent pieces onto one agent: an embedded autonomous BDI (`startAutonomousBDI`, identical code path to Agent B), a `createCoordinator` for talking to the partner, and a chat listener. Incoming chat messages are pushed through `createMissionQueue`, serializing mission handling to avoid races between rapid messages.

Pausing is *lazy*: the embedded BDI keeps harvesting until the first body-controlling tool (Table 7) is about to execute, via `pauseBdiForMission()` (idempotent). A pure rule/query mission therefore never interrupts ongoing play. `finally` unconditionally calls `bdi.resume()` at mission end, *unless* the agent was parked on an external signal (`partnerParkedOn`, Section 7), in which case the next mission turn must relay it first.

Pauses the embedded BDI		Does not pause it	
<code>collect_and_deliver,</code>	<code>go_to,</code>	<code>observe_environment,</code>	<code>calculate,</code>
<code>go_pick_up,</code>	<code>go_drop_off,</code>	every rule/navigation	store tool,
<code>rendezvous_with_partner,</code>		<code>direct_partner,</code>	<code>signal_partner</code>
<code>handoff_to_partner,</code>	<code>wait_for_partner</code>		

Table 7: BDI_PAUSING_TOOLS: only tools that take exclusive control of this agent’s own body (or must hold it still for a teammate) pause the embedded loop.

6.2 Prompt engineering

The system prompt (`SYSTEM_EXECUTOR_PROMPT`) is organized as a strict decision procedure, not free-form instructions:

Role / output contract. Exactly one tool call per step; the loop continues until `final_reply`.

Reward phrasing. Any mission tying points to an outcome (“+N pts if X”) is reframed as motivation: it resolves to a durable RULE or an active TASK, never a no-op.

Rejection whitelist. A *closed* list of four grounds — unresolved coordinate placeholder, explicit negative reward, violation of an active rule, or garbled text; everything else must be classified and acted on, and inventing further reasons (vagueness, missing specifics, future tense) is explicitly forbidden.

Anti-hallucination. The model may only use parcels/tiles/rewards present in the supplied state or a fresh `observe_environment` call — never invented.

Mission classification. Every mission is first sorted into RULE, TASK or QUERY; the prompt calls this “the most common mistake” and forces the decision before any tool call.

Compound missions. A boolean `more` flag on terminal tools lets “do X AND do Y” missions chain multiple terminal calls without ending after the first clause.

Team coordination rules. Eight numbered rules disambiguating solo vs. team missions, when to use `rendezvous_with_partner` versus manual `direct_partner` + `wait_for_partner`, and the asymmetric handoff protocol (Section 7).

Few-shot examples. `EXECUTOR_EXAMPLES` enumerates ~20 worked mission→tool-sequence pairs (rejections, factual queries, durable rules incl. compound and replacement cases, harvesting tasks, rendezvous, handoffs), each annotated with its reasoning.

6.3 Tool catalog

Category	Tools
Self-action	<code>move_to</code> , <code>pick_up_parcel</code> , <code>deliver_carried_parcel</code> , <code>collect_and_deliver</code> (delegates to the embedded BDI, Section 6.4)
Query	<code>observe_environment</code> , <code>calculate</code> , <code>resolve_delivery_tile</code>
Coordination	<code>rendezvous_with_partner</code> , <code>handoff_to_partner</code> , <code>direct_partner</code> , <code>signal_partner</code> , <code>wait_for_partner</code>
Persistent rules	<code>set_stack_size_rule</code> , <code>remove_stack_size_rule</code> , <code>set_parcel_value_rule</code> , <code>remove_parcel_value_rule</code> , <code>forbid_delivery_tile</code> , <code>prefer_delivery_tile</code> , <code>set_delivery_tile_multiplier</code> , <code>remove_delivery_tile_rule</code> , <code>clear_durable_rules</code> , <code>block_navigation_tile</code> , <code>unblock_navigation_tile</code> , <code>clear_navigation_rules</code>

Table 8: Tool catalog exposed to the executor LLM (`SYSTEM_EXECUTOR_TOOLS`).

6.4 Execution loop

For each chat message, a fresh `[system, user]` message pair is built — the system prompt plus a user prompt embedding the raw mission text, the rendered rule set, and a validator snapshot of the current game state. The loop (capped at `MAX_ITERATIONS=100`) repeats:

1. Call the model with `temperature=0` and the full tool list; strip the control-flow fields `thought` and `more` before forwarding parameters to the tool implementation.
2. `mapExecutorAction` translates the executor’s tool name/shape into the internal action used by `executeTool` (e.g. `pick_up_parcel` → `go_pick_up`).
3. `validateActionAgainstPersistentRules` checks the action against active rules *before* execution; a violation is fed back as a tool observation instead of running the disallowed action.
4. If the action is in `BDI_PAUSING_TOOLS`, pause the embedded BDI (once).
5. Execute the tool, append its observation as a `tool` message.
6. **Terminal tools** (every persistent-rule/navigation store, plus `collect_and_deliver` via `toolResult.terminal`) end the mission immediately on success, using the tool’s own confirmation text as the reply (saving a separate `final_reply` round-trip), *unless* the tool errored or `more:true` flags a remaining compound clause.
7. `final_reply` always ends the mission with an explicit message.

Concretely, for “collect three parcels and deliver them”: the LLM emits one `collect_and_deliver(parcel=3)` call; the embedded BDI is unpaused and watches `deliveredParcels` climb with zero further model calls until the target is reached (or the time budget elapses), ending the mission in a single round-trip.

7 Multi-Agent Coordination

Coordination is a thin asynchronous protocol over the SDK’s `say` channel (`utils/coordProtocol.js`), layered on top of, but not replacing, each agent’s ordinary intention queue.

Coordinator side (`llm/coordinator.js`). `pendingWaiters` (`cid` → `resolver`) and `bufferedStatuses` (`cid`

Message	Direction	Payload / effect
directive	LLM→BDI	{cid, command, args}; queued and, on arrival, preempts the BDI's running intention via <code>preemptForCoordination()</code> .
status	BDI→LLM	{cid, ok, detail}; resolves the matching <code>wait_for_partner</code> , or is buffered if none is waiting yet.
signal	LLM→BDI	{signal}; out-of-band release of <code>bs.coordination.waiting</code> .
rules	LLM→BDI	Full serialized rule snapshot (idempotent; a missed update self-heals on the next change), applied via <code>applyRulesSnapshot</code> .

Table 9: Coordination protocol messages (COORD_PROTO="coord").

→ status) together make `directPartner/waitForPartner` order-independent: a status that arrives before anyone waits for it is buffered and consumed by the next matching `waitForPartner`. `syncRules` fires automatically from `bs.rules.onChange`, so the BDI-only partner's copy of `bs.rules` never drifts from the LLM's.

BDI side (`bdi/coordination.js`). `setupBdiCoordination` attaches a listener that pushes directives into `bs.coordination.queue`, flips `active`, and calls `agent.preemptForCoordination()`; `#runCoordination()` then drains the queue one directive at a time via `coordToPredicate`, reusing the ordinary plan layer (Table 6) verbatim. `vacateAfterDrop()` steps the agent off its cell after a `putdown` so the partner can occupy it to collect the handoff parcel. A `resume` directive clears `coordination.active`; an idle TTL (`COORD_RESUME_IDLE_TTL_MS=15s`) auto-resumes defensively if the LLM partner disappears mid-directive.

Handoff protocol. A cross-delivery handoff is driven end-to-end by one tool, `handoff_to_partner`, instead of the model manually sequencing `direct_partner/wait_for_partner` calls: (1) it repeatedly issues `direct_partner(pickup)` with no coordinates so the BDI self-selects via `#selectBestPickup()` (reusing `intentionScore`, never diverging from normal play), up to a target count or safety cap (`HANDOFF_MAX_PICKUPS`); (2) `direct_partner(go_near)` brings the loaded BDI adjacent to the LLM agent; (3) `direct_partner(putdown)` drops at the BDI's cell, and `status(ok)` arrives only after `vacateAfterDrop()` steps it off the tile; (4) the LLM agent walks onto the vacated tile and retries the pickup; (5) by default it then delivers the load to the nearest delivery tile and reports what was actually banked.

Rendezvous / barrier synchronization. `rendezvous_with_partner` sends `go_near` to the partner, moves the LLM agent itself in parallel, then waits for both arrivals before releasing the partner with `resume` — a single atomic barrier that avoids the race window a manual `direct/move/wait` sequence would leave open.

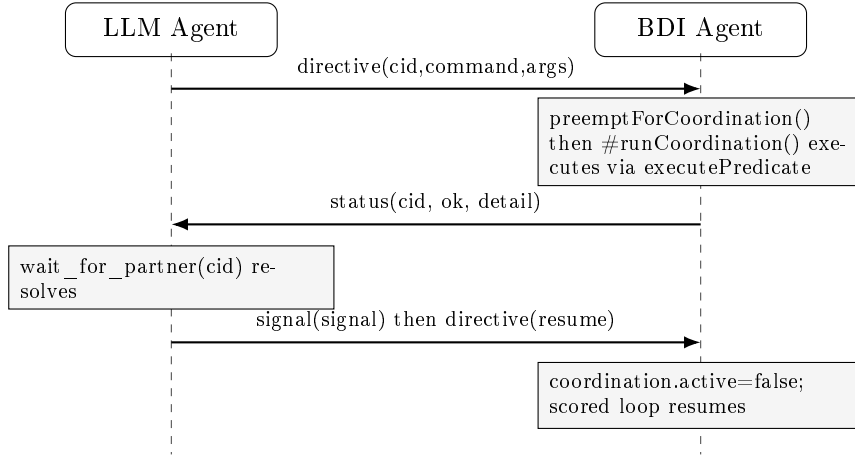


Figure 3: Multi-agent coordination protocol. Cross arrows are wire messages; boxed notes are local actions taken on receipt. The same exchange, with `pickup/putdown` commands, implements the asymmetric parcel handoff described above.

8 PDDL

When A* reports that a route is blocked and `crateBlocksRoute()` confirms a crate is the sole obstacle, the agent falls back to a STRIPS planner (`pddl/domain.pddl`) capable of pushing crates aside — a maneuver A* cannot express.

Actions: four `move-*` actions (precondition: the adjacent tile is free), and four matching `push-*` actions that move the agent onto the crate's tile while sliding the crate one step further onto a free `pushable` tile. Both collapse to the same primitive at execution time: the server has no separate push command, so moving *into* a crate with a clear type-5 tile behind it pushes it automatically (`planExecutor.js`'s `ACTION_MAP` maps every `push-*` to `actions.move(direction)`).

Predicate	Meaning
(tile ?t)	?t is a walkable cell (walls, type 0, are excluded entirely).
(free ?t)	No agent and no crate on ?t (parcels never block).
(delivery ?t), (pushable ?t)	Delivery zone / type-5 crate-receiving tile.
(up/down/left/right ?from ?to)	Directional adjacency between two existing tiles.
(at-agent ?t)	Current agent position.
(parcel ?p), (at-parcel ?p ?t), (carrying ?p), (delivered ?p)	Parcel type guard, floor position, carried state, terminal state.
(crate ?c), (at-crate ?c ?t)	Crate type guard and position.

Table 10: Domain predicates (`pddl/domain.pddl`).

Listing 1: Push action (excerpt, `pddl/domain.pddl`): the agent and the crate must be co-linear in the push direction, and the destination must be both **free** and **pushable**.

```

1 (:action push-right
2   :parameters (?c ?agentPos ?cratePos ?destPos)
3   :precondition (and (crate ?c) (at-agent ?agentPos)
4     (at-crate ?c ?cratePos) (right ?agentPos ?cratePos)
5     (right ?cratePos ?destPos)
6     (free ?destPos) (pushable ?destPos))
7   :effect (and (at-agent ?cratePos)
8     (not (at-agent ?agentPos)) (free ?agentPos)
9     (at-crate ?c ?destPos) (not (at-crate ?c ?cratePos))
10    (not (free ?destPos)))
11 )

```

Problem generation. `buildProblem(bs, goal)` (`pddl/problemBuilder.js`) caches the map’s static part (tiles, adjacency, delivery/pushable sets), rebuilding only on a new map; the dynamic part — occupancy, positions, parcel/crate locations — is always recomputed fresh. Tiles are named `t_<x>_<y>`; parcel/crate ids are hashed to a stable `[a-z0-9]` suffix (djb2 variant, `sanitize`) since PDDL object names cannot contain arbitrary characters and the mapping never needs inverting.

Goal type	Generated conjunct(s)
<code>reach_tile {x,y}</code>	<code>(at-agent t_x_y)</code>
<code>deliver_parcel {parcelId}</code>	<code>(delivered p_<hash>)</code>
<code>deliver_parcels {parcelIds}</code>	conjunction of <code>(delivered ...)</code> per id
<code>free_tile {x,y}</code>	<code>(free t_x_y)</code> — used to ask “can a crate be moved off this tile”, the goal driving the crate-recovery maneuver of Figure 4

Table 11: Goal descriptors accepted by `buildProblem`.

Planner invocation and fallback. `plan(bs, goal)` races `onlineSolver(domain, problem)` against a `SOLVER_TIMEOUT_MS=5s` deadline; an empty result and a timeout/solver error both normalize to `null`, so the caller falls back to ordinary A* either way. While a plan executes, `bs.committedManeuver=true` disables hysteresis preemption (Eq. 15), since a crate push deliberately walks *away* from the target first (to get behind the crate) and would otherwise look like a worse intention to the revision loop.

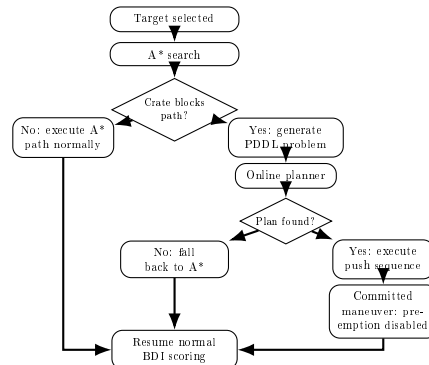


Figure 4: Crate blocking and PDDL recovery: `crateBlocksRoute()` double-checks the block before paying for a solver call; a found plan is a committed maneuver immune to preemption.